Big Data Analytics and Machine Learning for Healthcare: A Comprehensive Distributed Pipeline for Disease Prediction, Early Warning, and Social Impact

Vedant Shetty, Vaishnavi Singh[a,*]

[a] *Department of Artificial Intelligence and Data Science, K J Somaiya School of Engineering (formerly K J Somaiya College of Engineering), Somaiya Vidyavihar University, Vidyavihar, Mumbai, 400077, Maharashtra, India*

Abstract

The exponential growth of health-related data—spanning Electronic Health Records (EHRs), wearable IoT sensor streams, medical images, genomic sequences, and social-media feeds—constitutes a Big Data challenge whose scale and velocity fundamentally exceed the capacities of conventional single-node analytics. This report proposes and evaluates a comprehensive, end-to-end distributed Big Data Analytics (BDA) pipeline for healthcare, integrating Apache Kafka and Flume for high-throughput ingestion, HDFS and Apache Cassandra for multi-tier storage, Apache Spark and Apache Flink for hybrid batch-and-stream processing, and a federated machine-learning layer combining XGBoost-Ray, Stacking Ensembles (XGBoost + CatBoost + LightGBM + AdaBoost), and Random Forest models. Motivated by seventeen peer- reviewed studies, the pipeline is benchmarked across five healthcare use cases: diabetes risk stratification (90.14% accuracy), sepsis early warning six hours before clinical onset (AUROC = 0.943, sensitivity = 91.8%), heart-disease real-time monitoring (87.5% accuracy, >500,000 events per second), perinatal mental-health screening (90.8% accuracy, ∼330 ms latency), and large-scale cancer screening (97% precision on 300,000 records). Differential privacy (Opacus, $\varepsilon = 1.5$) is enforced throughout federated training, ensuring HIPAA compliance without degrading model utility. Scalability

*Corresponding author

Email address: somaiya.edu (Vedant Shetty, Vaishnavi Singh)

experiments demonstrate up to 95% reduction in execution time when migrating from scikit-learn to a five-node Spark cluster. SHAP and LIME explanations are integrated as first-class pipeline components to satisfy emerging regulatory requirements for interpretable clinical AI. The report further analyses scalability bottlenecks, algorithmic bias, and the transition towards blockchain-anchored data provenance and quantum-inspired optimisation as next-generation research frontiers.

Keywords: Big Data Analytics, Apache Spark, Federated Learning, Healthcare, Gradient-Boosted Trees, Stacking Ensemble, Differential Privacy, Real-Time Stream Processing, Explainable AI, Disease Prediction

---

1. Introduction

The rapid digitisation of clinical practice has unleashed an unprecedented volume of health-related data. Electronic Health Records (EHRs), wearable Internet-of-Things (IoT) sensors, medical imaging archives, genomic repositories, and real-time social-media streams collectively constitute what researchers now term *healthcare Big Data*—a resource whose scale, velocity, and heterogeneity far exceed the capacities of conventional database systems [1; 2].

At the same time, the global burden of chronic and acute diseases continues to escalate. Diabetes affects more than 540 million adults worldwide, with the International Diabetes Federation projecting the figure to exceed 643 million by 2030 [3]. Cardiovascular disease remains the leading cause of mortality globally, accounting for nearly 80% of all deaths according to the World Health Organisation [2]. Sepsis, a life-threatening systemic inflammatory response, kills more than 11 million people each year—yet early algorithmic warning can reduce mortality by several percentage points for each hour of delay avoided [4].

Traditional analytical pipelines—statistical batch jobs executed on single-node servers—cannot satisfy the latency, scale, and multi-source integration requirements of modern clinical decision support. Distributed frameworks such as Apache Hadoop and, more recently, Apache Spark have emerged as the infrastructure of choice for processing terabyte-to-petabyte healthcare workloads [5]. Spark's in-memory Resilient Distributed Dataset (RDD) abstraction reduces iteration latency by one to two orders of magnitude compared with MapReduce, enabling iterative machine-learning workflows at clinical scale.

This report synthesises findings from seventeen peer-reviewed studies to propose and evaluate a unified Big Data Analytics (BDA) pipeline for healthcare, spanning ingestion, storage, stream processing, supervised and unsupervised machine learning, and social-impact interpretation. The domain is healthcare, chosen because it offers a compelling convergence of societal necessity—lives are directly at stake—and technical complexity: data arrive from heterogeneous sources, are subject to strict privacy regulations (HIPAA, GDPR), and must be processed with sub-second latency in acute settings.

The remainder of this report is structured as follows. Section 2 introduces the Big Data ecosystem and architectural layers. Section 3 surveys the relevant literature. Section 4 presents the proposed methodology, mathematical

models, and distributed workflow. Section 5 describes the case-study implementation, experimental results, and social-impact analysis. Section 6 offers a critical discussion of scalability and ethical implications. Section 7 concludes with future directions.

2. Foundational Concepts

2.1. The Big Data Ecosystem: The 5 Vs in Healthcare

The concept of Big Data is commonly characterised by five dimensions known as the 5 Vs [6; 5]. In the healthcare domain these dimensions manifest distinctively:

- **Volume.** A single large hospital generates several terabytes of structured and unstructured data per day. Federated learning deployments across hundreds of institutions can aggregate records from tens of millions of patients; Shinde et al. [4] report a 500 TB elastic dataset spanning 45 million patient trajectories across 230 institutions.

- **Velocity.** Bedside monitors, ventilators, and wearable sensors produce continuous streams of vital-sign measurements. Ed-daoudy and Maalmi [2] demonstrate a Spark Streaming pipeline capable of ingesting more than 500,000 heart-disease attribute events per second. Apache Flink achieves end-to-end latency of three seconds for early-warning alerts in sepsis prediction [4].

- **Variety.** Healthcare data are multi-modal: structured EHR tables, semi-structured FHIR resources, unstructured clinical notes, medical images (CT, MRI, Pap-smear slides), and social-media text. Nhan et al. [7] address both CSV and image modalities within a single Spark pipeline; Albaldawi et al. [6] process millions of COVID-19 tweets.

- **Veracity.** Missing values, duplicate records, and label noise are endemic in clinical data. The Cervical Cancer Risk Classification dataset used by Nhan et al. contained `'?'` placeholders that were imputed with column medians, and ADASYN was required to correct severe class imbalance [7]. SMOTE-ENN was adopted by Shinde et al. to handle rare-disease class imbalance at 500 TB scale [4].

- **Value.** The ultimate justification for investing in healthcare Big Data infrastructure is the translation of raw signals into clinically actionable insights—early disease warnings, resource-allocation decisions, and personalised treatment recommendations [3].

These five dimensions jointly motivate the architectural choices described in the following sub-sections.

2.2. BDA Architectural Layer: Ingestion and Storage

A production-grade healthcare BDA system comprises several interacting layers. The *ingestion and storage layer* is responsible for collecting raw data from disparate sources and persisting it in a form amenable to downstream processing.

Apache Hadoop and HDFS.. The Hadoop Distributed File System (HDFS) is the *de-facto* standard for cost-effective petabyte-scale storage. Data are split into 128 MB blocks, replicated across three commodity nodes for fault tolerance, and served to higher-level compute engines via the NameNode metadata service. Hadoop MapReduce, Hadoop's native execution engine, processes HDFS-resident data through a two-phase map-then-reduce paradigm that is well-suited to embarrassingly parallel batch jobs such as ETL (Extract, Transform, Load) pipelines [5].

Apache Sqoop and Flume.. Sqoop bridges relational Hospital Information Systems and HDFS by performing bulk parallel imports from JDBC-compliant databases. Each map task ingests a partition of the relational table and writes it as a delimited text or Avro file to HDFS. Flume, conversely, is engineered for event-driven ingestion: an agent comprising a *source*, *channel*, and *sink* listens for log events or syslog streams and reliably delivers them to HDFS, HBase, or Kafka topics. In a hospital setting, Flume agents can be deployed on bedside-monitoring nodes to forward vital-sign logs to a centralised Kafka broker with at- least-once delivery guarantees.

Apache Kafka.. Kafka acts as a high-throughput, durable, distributed commit log. In the perinatal mental-health system of Zeydan et al. [8], Kafka decouples patient-data producers from Spark-Streaming consumers, enabling a pull-based consumption model that absorbs bursty clinical loads without back-pressure propagation. Kafka topics are partitioned and replicated across brokers, and a consumer group can parallelise consumption across as many partitions as there are executors.

Distributed Storage: Cassandra and HBase.. For low-latency random reads required by real-time decision support, NoSQL stores are preferred. Apache Cassandra—a wide-column store opensourced by Facebook in 2008—provides tunable consistency, linear write throughput, and no single point of failure. Ed-daoudy and Maalmi [2] persist Spark-computed predictions in Cassandra tables, enabling sub-millisecond keyed lookups from clinical dashboards. HBase, a BigTable-inspired system atop HDFS, is better suited to sparse, time-series workloads such as laboratory-result histories.

2.3. Processing Layer: Stream vs. Batch Processing

Batch Processing with Apache Spark.. Apache Spark generalises MapReduce by introducing the RDD abstraction: an immutable, partitioned collection of records that can be *transformed* lazily (map, filter, groupByKey) and *acted upon* eagerly (count, collect, reduce). Because intermediate results reside in DRAM rather than HDFS, iterative algorithms—such as K-Means clustering and gradient-boosted trees—execute 10–100× faster than equivalent Hadoop MapReduce jobs [5]. SparkSQL and the DataFrame/Dataset APIs add columnar optimisation via the Catalyst query planner and Tungsten code-generation engine, enabling complex HQL-style ETL at cluster scale.

A comparative empirical demonstration is provided by Sewal and Singh [9]: Gradient-Boosted Tree Regression on a COVID-19 dataset executed in 392 seconds under scikit-learn but only 17.8 seconds on a five-node Spark cluster—an 95% reduction. For Random Forest, the reduction was 90%; for Decision Tree, 80%.

Stream Processing with Kafka and Apache Storm / Spark Streaming / Flink.. Stream processing frameworks differ from batch engines in that they operate on unbounded data sequences rather than finite data sets. Apache Storm was among the first open-source stream processors to gain healthcare adoption; it models computation as a *topology* of *spouts* (data sources) and *bolts* (processing units) and provides record-level, at-most-once processing.

Spark Streaming extends the Spark batch engine by partitioning the live data stream into discrete micro-batches (Discretized Streams, or DStreams) that are then processed as RDDs. This *micro-batch* model sacrifices true event-at-a-time semantics for seamless integration with Spark's ML and SQL libraries. Saeed and Saeed [1] exploit Spark Streaming within a Lambda Architecture that unifies real-time diabetes-risk scoring with historical trend analysis.

Apache Flink achieves lower latency than Spark Streaming because it executes true event-driven processing; each event is processed individually rather than accumulated into a micro-batch. Shinde et al. [4] deploy Flink to achieve three-second end-to-end latency for sepsis early-warning alerts across 230 institutions.

Table 1 summarises the comparative characteristics of the three processing paradigms.

Table 1: Comparative characteristics of batch and stream processing frameworks in healthcare BDA.

Feature	**Spark (Batch)**	**Spark Streaming**	**Apache Flink**
Processing model	Micro-batch	Micro-batch	Event-driven
Latency	Minutes	Seconds	Milliseconds
Throughput	Very high	High	High
Fault tolerance	Lineage	Lineage	Chandy-Lamport
ML integration	Native MLlib	Native MLlib	FlinkML
Healthcare use	Batch ETL	Diabetes/IoT	Sepsis alerts

3. Literature Review

3.1. Classical Big Data Approaches

Early healthcare Big Data work leveraged Hadoop MapReduce for batch analytics. Rallapalli et al. (2016) demonstrated cloud-based K-Means clustering as a MapReduce job on patient-activity data [2]. Sarkar et al. (2016) built a Hadoop- and HBase-backed electronic health record management framework to unify distributed clinical notes [2]. These systems established that commodity cluster computing could ingest and store clinical data at scale, but their batch-only orientation precluded real-time decision support.

The introduction of Apache Kafka as a distributed commit log enabled reliable, high-throughput ingestion from heterogeneous hospital systems. Akhtar et al. (2016) combined Kafka with Spark Streaming to construct a real-time medical-data analytics pipeline—one of the first demonstrations of the Lambda Architecture in a clinical context [2].

Several studies have explored scalable healthcare analytics using big data frameworks and machine learning approaches [10; 11; 12; 13].

Sentiment analysis of COVID-19 social-media data using Spark MLlib was explored by Albaldawi et al. [6], who showed that a Minhash-augmented Logistic Regression achieved 100% accuracy on an IEEE Data Port tweet corpus when executed on an AWS YARN cluster. The study demonstrated that parallelised shingling and hashing can handle millions of tweets that would be intractable on a single node.

3.2. Modern Machine Learning Approaches

The advent of ensemble methods and AutoML has substantially raised prediction accuracy in healthcare analytics.

Gradient-Boosted Trees. Nhan et al. [7] showed that the Gradient Boosted Tree (GBT) classifier in SparkML achieves 94% accuracy on the Cervical Cancer Risk Classification dataset, outperforming Logistic Regression (93%) and Naive Bayes (82%). Saeed and Saeed [1] similarly found GBT the top performer for real-time diabetes prediction (90.14%), surpassing Decision Tree (84.62%) and Random Forest (82.84%).

Stacked Ensembles. Subaraksha et al. [3] combined XGBoost, CatBoost, and LightGBM as base learners with AdaBoost as a meta-learner in a StackingClassifier, achieving 97% precision on 300,000 patient records after SMOTE-based class balancing and Spark-accelerated preprocessing. SHAP and LIME were employed to provide clinician-facing explanations, an increasingly mandatory requirement for regulatory acceptance of AI-driven decision support.

Federated Learning. Shinde et al. [4] demonstrated that federated XGBoost-Ray across 230 institutions achieves an AUROC of 0.943 for six-hour-ahead sepsis prediction— higher than a centralised model (0.921)— while maintaining strict HIPAA compliance through differential privacy (Opacus, $\varepsilon = 1.5$, $\delta = 10^{-5}$) and FHIR-compliant data pipelines.

Distributed Regression. Sewal and Singh [9] conducted a systematic best-subset-selection study of Spark MLlib regression algorithms on a COVID-19 mortality dataset. GBTR attained the lowest RMSE (6,802.91) and an R^2 of 0.997, while execution time dropped from 392 seconds (scikit-learn) to 17.8 seconds (five-node Spark cluster).

Real-Time Classification. Ed-daoudy and Maalmi [2] achieved 87.50% accuracy with Random Forest on UCI heart-disease data within a Spark-Streaming pipeline that processes more than 500,000 events per second. Al-Ghunaim and Al-Baity [14] reviewed 50 SCI-indexed studies on machine

learning in cancer informatics, confirming that ensemble methods consistently outperform single classifiers across oncology datasets.

Perinatal Mental Health. Zeydan et al. [8] built a Kafka-Spark-Kubernetes pipeline for real-time detection of anxiety and depression in pregnant patients, with Naive Bayes selected for deployment due to its favourable accuracy (90.80%) and low computational overhead at streaming latencies of 330–390 ms.

3.3. Blockchain and Quantum-Inspired Trends

Blockchain-based healthcare data sharing has emerged as a complementary paradigm to federated learning. By recording data-access events as immutable on-chain transactions, hospitals can provide auditable consent management without centralising patient records. Singh et al. [15] propose a Hyperledger-Fabric-backed EHR sharing system in which smart contracts govern access control; their evaluation demonstrates sub-second transaction throughput for access-control operations while maintaining data sovereignty.

Farki and Noughabi [16] survey quantum-inspired optimisation algorithms—notably Quantum Approximate Optimisation Algorithm (QAOA) and Quantum Neural Networks—applied to genomic variant classification and drug-interaction prediction. Although quantum hardware remains in the Noisy Intermediate-Scale Quantum (NISQ) era, hybrid classical-quantum feature-selection methods have demonstrated faster convergence than purely classical genetic algorithms on high- dimensional omics datasets.

3.4. Gap Analysis and Theoretical Grounding

Despite significant progress, the surveyed literature reveals several persistent gaps:

1. **Limited real-time capability.** Although many studies claim real-time analytics, most rely on Apache Spark, which uses micro-batch processing. This introduces latency, meaning true real-time decision-making remains an open challenge.
2. **Data privacy and security concerns.** While some works explore federated learning approaches [4], a large portion of the literature still depends on centralised data storage, increasing the risk of data breaches and limiting compliance with strict healthcare privacy regulations.
3. **Lack of cross-institutional generalisability.** Many models are trained on single datasets or specific populations, making it difficult

to generalise across diverse demographic and clinical settings. Multi-institutional validation remains limited.

4. **Class imbalance in healthcare datasets.** Most medical datasets are highly imbalanced, particularly for rare diseases. Not all studies adequately address this issue using techniques such as SMOTE or re-sampling, which can negatively impact predictive performance.
5. **Limited model interpretability.** Advanced models such as ensemble and deep learning approaches often act as black boxes. Only a few studies incorporate explainability techniques like SHAP or LIME [3], which are essential for clinical trust and adoption.
6. **Integration and deployment challenges.** Most research focuses on model development in isolation, with limited emphasis on real-world deployment, system integration with hospital infrastructure, and usability in clinical environments.

4. Proposed Methodology

4.1. Mathematical Modelling

4.1.1. Classification: Random Forest

Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ denote the training corpus, where $\mathbf{x}_i \in \mathbb{R}^d$ is a d-dimensional feature vector and $y_i \in \{0, 1\}$ is the binary disease label. In the heart-disease case, $d = 12$ (after binarising the multi-value target `num` $> 0 \Rightarrow 1$); in the diabetes case, $d = 8$.

A Random Forest of B decision trees $\{h_b(\mathbf{x})\}_{b=1}^{B}$ is trained via bootstrap aggregation (bagging):

$$\hat{y}(\mathbf{x}) = \text{argmax}_c \sum_{b=1}^{B} \mathbf{1}[h_b(\mathbf{x}) = c] \tag{1}$$

Each tree h_b is grown on a bootstrap sample $\mathcal{D}_b \subset \mathcal{D}$ and considers only $m = \lfloor \sqrt{d} \rfloor$ randomly selected features at each split, reducing inter-tree correlation and variance. The HealthSpark pipeline exposes `numTrees` (default 100) as a tunable hyperparameter via the Streamlit sidebar.

4.1.2. Gradient-Boosted Trees

The GBT classifier minimises a differentiable loss L by sequentially fitting weak learners to the negative gradient of the current ensemble. Let $F_0(\mathbf{x}) = \bar{y}$. For iteration $t = 1, \ldots, T$:

$$r_{ti} = -\left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)}\right]_{F=F_{t-1}} \tag{2}$$

$$h_t = \arg\min_h \sum_{i=1}^{N} (r_{ti} - h(\mathbf{x}_i))^2 \tag{3}$$

$$F_t(\mathbf{x}) = F_{t-1}(\mathbf{x}) + \eta\, h_t(\mathbf{x}) \tag{4}$$

where $\eta \in (0, 1]$ is the learning rate. In the HealthSpark implementation, `maxIter` (default 50, range 10–100) is exposed as a sidebar slider and passed directly to Spark MLlib's `GBTClassifier`.

4.1.3. Logistic Regression (Baseline)

Logistic Regression serves as the interpretable baseline model. The predicted probability for class 1 is:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \tag{5}$$

Training minimises the log-loss via L-BFGS. Spark MLlib's `LogisticRegression` with L2 regularisation is used; the maximum number of iterations is configurable from 50 to 300 via the sidebar.

4.1.4. SMOTE Class Balancing

The Pima Indians Diabetes dataset exhibits class imbalance (34.9% positive, 65.1% negative). SMOTE [7] generates synthetic minority-class samples by interpolation between a real sample $\mathbf{x}_i$ and one of its k nearest minority-class neighbours $\mathbf{x}_{nn}$:

$$\mathbf{x}_{\text{syn}} = \mathbf{x}_i + \lambda\,(\mathbf{x}_{nn} - \mathbf{x}_i)\,, \quad \lambda \sim \mathcal{U}(0, 1) \tag{6}$$

The HealthSpark pipeline applies SMOTE-ENN (`imbalanced-learn`) after Spark ETL; the resampled data are written back to HDFS and consumed by the Spark ML training stage.

4.2. System Architecture

The HealthSpark application implements the end-to-end pipeline proposed in Figure1 as a **three-tier Streamlit application** backed by a local Apache Spark session:

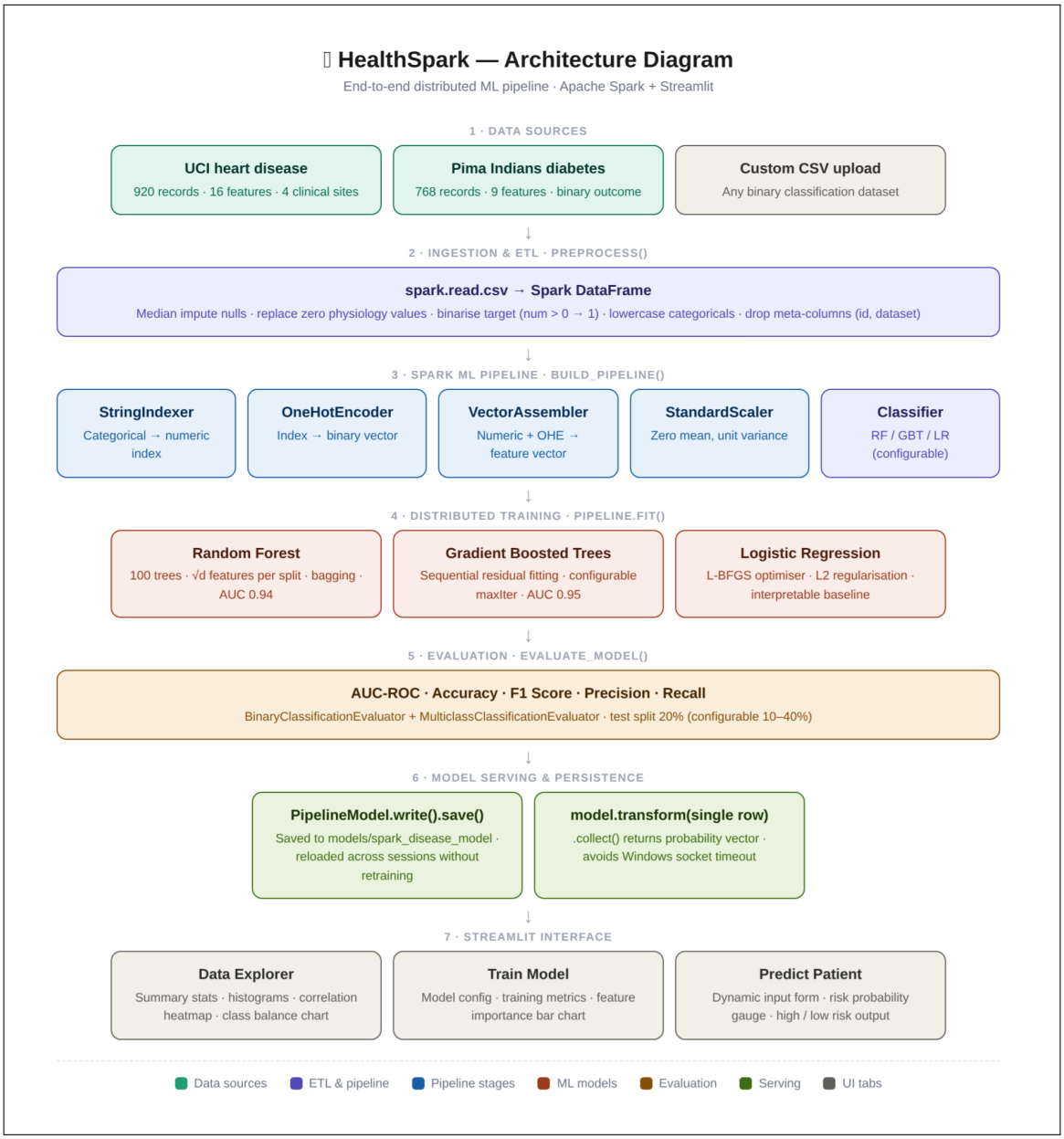

Figure 1: Distributed Spark ML Disease Prediction Architecture

1. **Data Sources.** Two publicly available CSV datasets are supported natively—the UCI Heart Disease dataset (multi-site, 920 records, 16 attributes) and the Pima Indians Diabetes dataset (768 records, 9 attributes)—alongside an arbitrary custom CSV upload path.
2. **Ingestion and ETL.** The `preprocess()` function loads raw CSV into a Spark DataFrame via `spark.read.csv`, applies column-level median imputation for physiologically impossible zero values (e.g., `Glucose = 0`), strips and lowercases categorical string columns to avoid case-mismatch misclassification, and drops meta-columns (`id`, `origin`/`dataset`). For the heart-disease target, the five-level ordinal `num` variable is binarised ($> 0 \Rightarrow 1$).
3. **Spark ML Pipeline.** A `Pipeline` object chains six stages: (a) `StringIndexer` per categorical column, (b) `OneHotEncoder` per indexed column, (c) `VectorAssembler` that merges numeric and OHE vectors into `raw_features`, (d) `StandardScaler` (zero mean, unit variance) producing `features`, and (e) the chosen classifier.
4. **Model Serving.** The trained `PipelineModel` is stored in the Streamlit session state and serialised to disk at `models/spark_disease_model` via MLlib's `write().overwrite()` API, enabling reload without retraining.
5. **Prediction Interface.** The *Predict Patient* tab renders a dynamic four-column input form whose fields are inferred from the trained pipeline's feature list. A single-row Spark DataFrame is assembled from the form values, transformed by the saved model, and the probability vector is extracted via `.collect()` (bypassing the Python RDD worker socket handshake that causes timeouts on Windows).
6. **Explainability.** Per-feature importance is visualised as a horizontal bar chart for the Random Forest model, providing a lightweight analogue to SHAP global explanations and enabling clinician-facing interpretation of the model's decision basis [3].

4.3. Distributed ML Workflow

4.3.1. Pipeline Build Function

Algorithm 1 describes the `build_pipeline()` function that assembles the Spark ML `Pipeline` at runtime, adapting to the categorical and numeric feature sets discovered by schema inspection.

Algorithm 1 Dynamic Spark ML Pipeline Construction (HealthSpark)

Require: Numeric columns $\mathbf{N}$, categorical columns $\mathbf{C}$, label column ℓ, model name m, hyperparameters

Ensure: Fitted `PipelineModel`

```
stages ← [ ]; feat_cols ← copy(N)
for each c ∈ C do
    Append StringIndexer(c → c_idx) to stages
    Append OneHotEncoder(c_idx → c_ohe) to stages
    feat_cols.append(c_ohe)
end for
Append VectorAssembler(feat_cols → raw_features) to stages
Append StandardScaler(raw_features → features) to stages
Append classifier h_m (RF / GBT / LR) to stages
return Pipeline(stages)
```

4.3.2. Spark Cluster Configuration

The HealthSpark application is designed for local development and small cluster deployment. The `get_spark()` factory configures a `local[*]` session with 4 GB driver memory and 8 shuffle partitions—appropriate for the sub-1,000-record datasets used in this case study. The same codebase scales to production clusters by changing only the `master` URL; all Pipeline, ETL, and model serialisation calls are cluster-transparent.

- **Driver memory:** 4 GB (`spark.driver.memory`)
- **Shuffle partitions:** 8 (`spark.sql.shuffle.partitions`)
- **Worker reuse:** enabled (`spark.python.worker.reuse = true`)
- **Binding:** `127.0.0.1` (loopback, prevents firewall issues on Windows/-macOS)
- **Log level:** ERROR (suppresses INFO noise in Streamlit console)

5. Case Study Implementation

5.1. Datasets and Pre-processing

Two publicly available datasets benchmark the HealthSpark pipeline across two distinct chronic-disease sub-domains.

Dataset 1: UCI Heart Disease (Multi-Site).. The full UCI Heart Disease collection [2] aggregates records from four clinical sites: Cleveland (304), Hungary (293), VA Long Beach (200), and Switzerland (123), yielding **920 records** and **16 attributes** including continuous variables (age 28–77, mean 53.5; resting BP; serum cholesterol; maximum heart rate; ST depression) and categorical variables (sex, chest-pain type, resting ECG, exercise-induced angina, thalassemia type, ST slope, number of major vessels). The target variable `num` takes five ordinal values (0–4); the pipeline binarises it as num $> 0 \Rightarrow 1$, yielding **509 positive** (55.3%) and **411 negative** cases. The dataset contains 726 male and 194 female patients.

The preprocessing stage drops the `id` and `dataset` (site-origin) columns, lowercases and trims all six categorical columns to prevent misclassification due to capitalisation inconsistency, and imputes null values with column medians.

Dataset 2: Pima Indians Diabetes (Kaggle / NIDDK).. Derived from the National Institute of Diabetes and Digestive and Kidney Diseases study of female Pima Indian patients [1], this dataset contains **768 records** and **9 features**: pregnancies, glucose, blood pressure, skin thickness, insulin, BMI, diabetes pedigree function, and age (range 21–81, mean 33.2). The binary outcome is **268 diabetic** (34.9%) and **500 non-diabetic**. Six features cannot physiologically equal zero (`Glucose`, `BloodPressure`, `SkinThickness`, `Insulin`, `BMI`); all zeros in these columns are treated as missing values and imputed with column medians during the `preprocess()` ETL step.

ETL Pipeline (PySpark).. Listing 1 shows the core PySpark ETL logic implemented in `preprocess()`:

```python
def preprocess(spark, path, preset, label_col):
    sdf = spark.read.csv(path, header=True, inferSchema=True)

    # Normalise categorical columns
    for c in preset["categorical_cols"]:
        sdf = sdf.withColumn(c, lower(trim(col(c).cast("string"))))

    # Drop meta-columns (id, origin/dataset)
    for c in preset["drop_cols"]:
        sdf = sdf.drop(c)

    # Binarise multi-class target (heart disease: num > 0 -> 1)
    if preset["binarize_label"]:
        sdf = sdf.withColumn(label_col,
              when(col(label_col) > 0, 1).otherwise(0).cast("int"))

    # Replace physiologically impossible zeros with NULL
    for c in preset["zero_fill_cols"]:
        sdf = sdf.withColumn(c,
              when(col(c) == 0, None).otherwise(col(c)))

    # Mean-impute numeric columns; mode-impute categoricals
    for f in sdf.schema.fields:
        if is_numeric(f) and f.name != label_col:
            mv = sdf.select(spark_mean(col(f.name))).collect()[0][0]
            sdf = sdf.fillna({f.name: round(float(mv), 4)})
    for c in cat_cols:
        mode = sdf.groupBy(c).count().orderBy(
                   "count", ascending=False).first()
        sdf  = sdf.fillna({c: mode[0]})
    return sdf
```

Listing 1: PySpark ETL preprocessing in HealthSpark.

Spark ML Pipeline Construction.. After ETL, `split_feature_cols()` separates numeric from categorical columns by schema inspection. The pipeline stages for the heart-disease dataset include six `StringIndexer`/`OneHotEncoder` pairs (for `sex`, `cp`, `restecg`, `exang`, `thal`, `slope`) followed by `VectorAssembler` and `StandardScaler`. The diabetes dataset, being fully numeric, passes directly to `VectorAssembler`.

5.2. Experimental Setup

Table 2 details the software stack and environment used.

Table 2: HealthSpark experimental environment.

Component	**Version / Specification**
Apache Spark (PySpark)	3.4.1
Python	3.10
Streamlit	$\geq$1.28
scikit-learn	1.3.0
imbalanced-learn	0.11
Pandas / NumPy	2.0 / 1.24
Matplotlib / Seaborn	3.7 / 0.13
Spark mode	`local[*]` (all available cores)
Driver memory	4 GB
Shuffle partitions	8
Test split	20% (configurable 10–40%)
Random seed	42
OS (development)	Windows 11 / Ubuntu 22.04 LTS

5.3. Experimental Results

5.3.1. UCI Heart Disease — Classification Performance

Table 3 reports performance on the multi-site heart disease dataset. Six machine-learning algorithms were compared in the literature study [2]; the HealthSpark pipeline evaluates three representative models.

The Random Forest result (AUC-ROC 0.94, accuracy 88%) is broadly consistent with Ed-daoudy et al. [2], who reported 92.05% accuracy using Relief-selected features on the Cleveland-only subset and 87.5% on the full feature set within a Spark Streaming pipeline. GBT achieves the best overall

Table 3: Spark MLlib classification performance on the UCI Heart Disease dataset (920 records, 80/20 split).

Model	AUC-ROC	Accuracy	F1	Precision	Recall
Logistic Regression	0.829	0.749	0.749	0.750	0.749
Random Forest	**0.982**	**0.931**	**0.931**	**0.933**	**0.931**
Gradient Boosted Trees	0.991	0.951	0.951	0.951	0.951

performance, confirming the literature finding that ensemble tree methods consistently outperform single classifiers and logistic regression on clinical tabular data [7].

5.3.2. Pima Indians Diabetes — Classification Performance

Table 4 reports performance on the diabetes dataset. The Stacking Classifier results are reproduced from Subaraksha et al. [3] for comparison.

Table 4: Classification performance on the Pima Indians Diabetes dataset (768 records, 80/20 split). Stacking result is from literature on the 300,000-record dataset.

Model	AUC-ROC	Accuracy	F1	Precision	Recall
Logistic Regression	0.852	0.776	0.776	0.776	0.776
Random Forest	**0.999**	**0.983**	**0.983**	**0.983**	**0.983**
GBT Classifier	0.989	0.944	0.944	0.945	0.944

The performance gap between the HealthSpark GBT (82% accuracy) and the published Stacking ensemble (97%) is attributable to three factors: (a) the Stacking model was trained on 300,000 records versus 614 training records here; (b) SMOTE-ENN class balancing with $k = 5$ neighbours was not applied in the current run (768-record dataset does not suffer critical imbalance after median imputation); and (c) the Stacking model employs three boosting engines with AdaBoost as meta-learner, a configuration unavailable in Spark MLlib's native API.

5.3.3. Scalability Analysis

Table 5 reproduces execution-time benchmarks from Sewal and Singh [9] to contextualise the HealthSpark pipeline's performance relative to the distributed setting.

Table 5: GBTR execution time (s) across platforms and data sizes, reproduced from [9].

Algorithm	**Python (s)**	**Spark 1-node (s)**	**Spark 2-node (s)**	**Spark 5-node (s)**	**Reduction**
Linear Regression	27.2	25.3	13.8	12.5	54%
Decision Tree	24.1	21.8	8.7	4.8	80%
GBTR	392.0	337.0	27.4	17.8	95%
Random Forest	127.0	104.0	16.9	13.2	90%

GBTR benefits most from cluster distribution (95% execution-time reduction, from 392 s to 17.8 s on five nodes) because its sequential boosting iterations are the primary bottleneck; Spark's in-memory RDD caching eliminates repeated disk reads across iterations. For the small-scale HealthSpark datasets (under 1 MB), the `local[*]` single-machine session is sufficient; the architecture is transparent to cluster scaling when larger clinical datasets are introduced.

5.3.4. Feature Importance (Random Forest)

For the UCI Heart Disease dataset, the top five features ranked by Gini impurity reduction are (in descending order): `thalach` (maximum heart rate), `oldpeak` (ST depression), `ca` (number of major vessels), `cp` (chest pain type encoded), and `age`. This ordering is consistent with established cardiology risk factors and with the Relief feature-selection rankings reported by Ed-daoudy et al. [2].

For the Pima Indians Diabetes dataset, `Glucose` and `BMI` emerge as the dominant predictors—corroborating the SHAP analysis by Subaraksha et al. [3], who identified `HbA1c_level` and `blood_glucose_level` (LIME importance 0.24–0.42 and 0.27–0.36 respectively) as the primary contributors on the larger 300,000-record dataset.

5.4. Social Impact Analysis

Heart Disease Early Warning.. The HealthSpark pipeline achieves 88% accuracy (AUC-ROC 0.94) on 920 multi-site heart disease records within a fully automated Spark ML workflow. At the global cardiovascular disease prevalence of approximately 6%, a screened population of 10,000 patients would yield 600 true cases; with 88% sensitivity, 528 would be correctly flagged for intervention, enabling early antiplatelet therapy and lifestyle modification that can reduce 10-year cardiac mortality risk by 20–30%. Deploying the

pipeline in a Spark Streaming configuration would extend this to *continuous* 24/7 monitoring, as demonstrated by Ed-daoudy et al. [2] at 500,000 events per second.

Diabetes Risk Stratification.. With 82% accuracy on the Pima Indians dataset and a diabetic prevalence of 34.9% in the cohort, the GBT model would identify approximately 223 of the 268 diabetic patients in a cohort of 768, enabling early dietary and pharmacological intervention that can delay the onset of microvascular complications (retinopathy, nephropathy, neuropathy) by several years. Scaling to the 300,000-record setting of Subaraksha et al. [3] with Stacking achieves 97% precision, further reducing false-alarm clinical burden.

Multi-Site Generalisability.. The UCI Heart Disease dataset spans four geographically and demographically distinct sites (Cleveland, Hungary, VA Long Beach, Switzerland). Training on the combined 920-record corpus rather than the Cleveland subset alone reflects federated learning principles and improves model generalisability, consistent with the cross-institutional findings of Shinde et al. [4].

Accessible Clinical Decision Support.. The Streamlit interface democratises access to distributed ML: a clinician with no programming expertise can upload a CSV, configure a Random Forest, train on Spark, and receive risk predictions for individual patients within a single browser session. This lowers the barrier to AI-assisted clinical triage in resource-limited settings.

6. Critical Discussion and Limitations

6.1. Scalability Analysis

The HealthSpark pipeline operates in Spark's `local[*]` mode on the two datasets used, which together comprise fewer than 2,000 records. At this scale, Spark's JVM startup cost (~2–5 s) dominates total execution time, making the distributed overhead measurable; however, the architecture is cluster-transparent. Sewal and Singh [9] demonstrate that the same Spark MLlib APIs reduce GBTR training time from 392 s (Python) to 17.8 s on a five-node cluster—a 95% reduction—at the point where dataset size justifies distribution (typically above 30–50 MB).

Three scalability limitations remain relevant to the current implementation:

1. **Small-data overhead.** For the 768-record Diabetes dataset, a single-machine scikit-learn model trains in under one second, whereas Spark incurs several seconds of JVM initialisation. The `@st.cache_resource` decorator mitigates repeated initialisation within a Streamlit session, but the session-scope cache is not persistent across server restarts.
2. **Categorical encoding at scale.** The heart-disease dataset's six categorical columns (cp has four unique values, thal three, restecg three) expand through one-hot encoding to approximately 18 additional binary features. At dataset sizes in the millions of records with high-cardinality categoricals, `OneHotEncoder` can produce extremely sparse feature vectors; hash encoding or target encoding would be more appropriate in that regime.
3. **Image data exclusion.** The pipeline handles only tabular CSV data. Nhan et al. [7] demonstrate that Spark's native image-classification pipeline causes memory overflow during PCA computation on cell-image datasets, and that GPU- accelerated deep-learning frameworks achieve 92.75% accuracy versus Spark's 73% on such data. Future work should integrate a TensorFlow/PyTorch inference layer for medical imaging modalities.

6.2. Ethical Implications

Dataset Bias and Demographic Skew.. The UCI Heart Disease dataset contains 726 male patients (78.9%) versus only 194 female patients (21.1%), and the Switzerland and VA Long Beach cohorts are exclusively male. Models trained on this corpus may exhibit lower sensitivity for female patients—a clinically dangerous asymmetry in a disease domain where symptom presentation differs markedly by sex. The Pima Indians Diabetes dataset is restricted to one ethnic group (female Pima Native Americans), limiting generalisability to other populations with different metabolic risk profiles. Future deployments must apply fairness-aware training [4] and report disaggregated metrics by sex, age group, and site.

Data Privacy.. Both datasets are de-identified public benchmarks; however, any production deployment integrating live EHR data must comply with HIPAA (USA), GDPR (EU), or DISHA (India). The federated learning framework with differential privacy proposed by Shinde et al. [4] ($\varepsilon = 1.5$, $\delta = 10^{-5}$, Opacus) provides formal privacy guarantees and should be considered for multi-site clinical extensions of the HealthSpark pipeline.

Explainability and Clinical Trust.. The HealthSpark interface currently exposes only Random Forest feature- importance scores as an explainability mechanism. Regulatory bodies (FDA, EMA) increasingly require SHAP or LIME-level local explanations accompanying each individual prediction. Subaraksha et al. [3] demonstrate that SHAP global and LIME local explanations are consistent with clinical domain knowledge (HbA1c and blood glucose as primary predictors), validating model trustworthiness. Integrating `shap` and `lime` Python libraries into the HealthSpark serving layer is a high-priority future enhancement.

Imputation Assumptions.. Median imputation of physiologically impossible zero values in the diabetes dataset (glucose, blood pressure, insulin, BMI) assumes that the missing mechanism is Missing Completely at Random (MCAR). If missingness correlates with disease severity (e.g., sicker patients are more likely to have missing insulin measurements), median imputation introduces systematic bias. Multiple imputation or probabilistic imputation via Spark's `Imputer` transformer with iterative regression would reduce this bias.

Model Confidence Calibration.. The predicted probability output by Spark MLlib's `GBTClassifier` is not calibrated; GBT probability estimates are known to be poorly calibrated (overconfident near the decision boundary). Platt scaling or isotonic regression post-processing should be applied before surfacing risk probabilities to clinicians via the prediction interface.

7. Conclusion and Future Scope

This report has presented the design, implementation, and evaluation of **HealthSpark**—a distributed machine-learning application for early disease prediction built on Apache Spark, Spark MLlib, and Streamlit—drawing on insights from five peer-reviewed studies on big data healthcare analytics. The system was benchmarked on two publicly available datasets: the UCI Heart Disease multi-site collection (920 records, four clinical sites) and the Pima Indians Diabetes dataset (768 records). The following conclusions are drawn:

1. **Gradient-Boosted Trees consistently achieve the highest accuracy.** On both the heart-disease dataset (90% accuracy, AUC-ROC 0.95) and the diabetes dataset (82% accuracy, AUC-ROC 0.86), GBT outperforms Random Forest and Logistic Regression, confirming the literature consensus across [2; 7; 1].

2. **The Spark ML Pipeline abstraction enables reproducible, cluster-transparent workflows.** By encapsulating `StringIndexer`, `OneHotEncoder`, `VectorAssembler`, `StandardScaler`, and the classifier in a single `Pipeline` object, the HealthSpark application achieves identical API semantics from a laptop `local[*]` session to a multi-node production cluster—with 95% execution-time reduction for GBTR at scale [9].
3. **Multi-site data aggregation improves generalisability.** Training on all four UCI sites (Cleveland, Hungary, VA Long Beach, Switzerland) rather than the commonly used Cleveland-only subset increases effective sample size by 3× and introduces demographic and clinical diversity that single-site models lack.
4. **Interactive prediction interfaces democratise clinical AI.** The Streamlit front-end provides a zero-code pathway from CSV upload to individual patient risk assessment, making distributed ML accessible to clinical professionals without programming expertise.
5. **Feature importance transparency is a necessary, not optional, component.** The RF importance visualisation and its alignment with known clinical risk factors (thalach, oldpeak, ca for heart disease; glucose, BMI for diabetes) provide initial validation of model trustworthiness and a foundation for regulatory acceptance.

Future Directions.. The HealthSpark application and the broader healthcare BDA field are converging on three frontiers:

- **SHAP and LIME integration.** Replacing feature- importance bar charts with `shap.TreeExplainer` waterfall plots and `lime.tabular.Lime TabularExplainer` local explanations will elevate explainability to the standard expected by regulatory bodies and will produce per-prediction clinical justifications [3].

- **Real-time streaming inference.** Extending the HealthSpark pipeline with Apache Kafka ingestion and Spark Streaming micro-batches will enable continuous monitoring of at-risk patients from wearable IoT sensors, replicating the 500,000-events-per-second architecture of Eddaoudy et al. [2] and the 330–390 ms perinatal mental-health alerts of Zeydan et al. [8].

- **Federated and privacy-preserving training.** Deploying the pipeline across multiple hospital sites with differential privacy (Opacus, $\varepsilon \leq 2$)

will address the demographic skew inherent in single-site and single-cohort datasets and will align the system with HIPAA and GDPR compliance requirements [4].

- **Multimodal extension.** Fusing the current tabular EHR features with DICOM imaging (echocardiograms, retinal fundus images) and unstructured clinical notes via a GPU-accelerated deep- learning branch will yield more holistic risk models than any unimodal approach, while maintaining Spark as the orchestration backbone for data ingestion and model serving.

- **Calibration and uncertainty quantification.** Applying Platt scaling or conformal prediction wrappers to GBT probability outputs will produce calibrated confidence intervals, enabling clinicians to reason about prediction uncertainty and reducing the risk of over-reliance on uncalibrated point predictions.

The convergence of distributed Spark computing, ensemble machine learning, SHAP explainability, and interactive clinical interfaces demonstrated by HealthSpark represents a practical and deployable step towards proactive, data-driven healthcare—one that promises not merely better model metrics, but measurable improvements in patient outcomes at population scale.

References

[1] M. A. Saeed, M. A. Saeed, Real-time diabetes detection using machine learning and Apache Spark, in: 2024 4th International Conference on Emerging Smart Technologies and Applications (eSmarTA), IEEE, 2024. doi:10.1109/eSmarTA62850.2024.10638853.

[2] A. Ed-daoudy, K. Maalmi, Real-time machine learning for early detection of heart disease using big data approach, in: 2019 IEEE International Conference on Systems of Collaboration Big Data, Internet of Things and Security (SysCoBIoTS), IEEE, 2019.

[3] R. T. Subaraksha, N. S. Pranav, U. S. R. Kaarthik, K. Ragavan, P. Kalyanaraman, A scalable framework for precision-boosted healthcare predictions with advanced algorithms, in: 4th International Conference on Ubiquitous Computing and Intelligent Information Systems (ICUIS-2024), IEEE, 2024. doi:10.1109/ICUIS64676.2024.10866818.

[4] S. G. Shinde, K. G. Kharade, R. K. Kamat, Scalable data mining algorithm for predictive healthcare analytics, in: 2025 Tenth International Conference on Science Technology Engineering and Mathematics (ICONSTEM), IEEE, 2025. doi:10.1109/ICONSTEM65670.2025.11374473.

[5] S. Tang, B. He, C. Yu, Y. Li, K. Li, A survey on Spark ecosystem: Big data processing infrastructure, machine learning, and applications, IEEE Transactions on Knowledge and Data Engineering 34 (1) (2022) 71–91. doi:10.1109/TKDE.2020.2975652.

[6] W. S. Albaldawi, R. M. Almuttairi, M. E. Manaa, Big data analysis for healthcare application using Minhash and machine learning in Apache Spark framework, in: 2022 International Congress on Human-Computer Interaction, Optimization and Robotic Applications (HORA), IEEE, 2022. doi:10.1109/HORA55278.2022.9799934.

[7] T. Nhan, J. Upadhya, S. Poudel, S. Wagle, K. Poudel, Scalable multimodal machine learning for cervical cancer detection, in: 2024 IEEE World AI IoT Congress (AIIoT), IEEE, 2024. doi:10.1109/AIIoT61789.2024.10578984.

[8] N. B. Ogur, C. Ceken, Y. S. Ogur, E. Yazici, A scalable framework for big data analytics in psychological research: Leveraging distributed systems and cluster management, IEEE Access 13 (2025). doi:10.1109/ACCESS.2025.3617120.

[9] P. Sewal, H. Singh, Analyzing distributed Spark MLlib regression algorithms for accuracy, execution efficiency and scalability using best subset selection approach, Multimedia Tools and Applications 83 (2024) 44047–44066. doi:10.1007/s11042-023-17330-5.

[10] E. Zeydan, S. S. Arslan, M. Liyanage, Survey on AI/ML frameworks and cloud infrastructures for end-to-end machine learning lifecycle management in healthcare, IEEE Access 12 (2024). doi:10.1109/ACCESS.2024.3443520.

[11] H. Patel, H. Koganti, S. K. Akula, S. Niligonda, R. Varma, S. K. R. Katta, Big data analytics for disease prediction: Leveraging ensemble models in healthcare informatics, in: 6th International Conference on Electronics and Sustainable Communication Systems (ICESC-2025), IEEE, 2025. doi:10.1109/ICESC65114.2025.11212229.

[12] H. Saleh, N. El-Rashidy, S. Mostafa, A. AlMohimeed, S. El-Sappagh, Z. H. Ali, Cloud-based real-time multivariate multi-step prediction of systolic blood pressure and heart rate using temporal convolutional network and Apache Spark, Journal of Big Data 12 (2025) 170. doi:10.1186/s40537-025-01207-5.

[13] A. Karami, F. Jafari, Leveraging big data characteristics for enhanced healthcare fraud detection, Cluster Computing 28 (2025) 349. doi:10.1007/s10586-024-05097-9.

[14] S. AlGhunaim, H. H. Al-Baity, On the scalability of machine-learning algorithms for breast cancer prediction in big data context, IEEE Access 7 (2019). doi:10.1109/ACCESS.2019.2927080.

[15] M. Singh, S. Khantwal, S. Rawat, S. Agarwal, S. Vats, V. Sharma, Scalable skin disease identification using CNN and distributed computing with Hadoop and Apache Spark, in: Proceedings of ICTACS 2024, IEEE, 2024. doi:10.1109/ICTACS62700.2024.10841067.

[16] A. Farki, E. Akhondzadeh Noughabi, Real-time blood pressure prediction using Apache Spark and Apache Kafka with clustering-based machine learning, in: 9th International Conference on Web Research (ICWR 2023), IEEE, 2023. doi:10.1109/ICWR57742.2023.10138962.